

Un agente Deep Q-Network per il gioco Knister

Francesco Simbola — matricola 65370

Università degli Studi di Cagliari

Reinforcement and Adaptive Learning — Prof. Alessandro Sebastian Podda

Luglio 2026

Abstract

Questo documento descrive lo sviluppo di un agente Deep Q-Network per Knister, un gioco di piazzamento dadi su griglia 5×5 con orizzonte fisso di 25 mosse. Il lavoro si articola in tre fasi. Nella prima, a partire da un'implementazione funzionante che raggiungeva un punteggio medio di $47,08 \pm 7,85$ su 10.000 partite hold-out, vengono individuati e corretti due difetti nel codice. Nella seconda viene condotto uno studio di ablazione sistematico su sei varianti algoritmiche: i ritorni a n passi ($n = 3$) si rivelano la leva dominante e portano da soli il punteggio a $58,90 \pm 9,42$, risultato che si dimostra stabile su due semi e due larghezze di rete. Nessuna delle altre cinque varianti supera questo limite, che risulta quindi riproducibile per l'architettura completamente connessa. Nella terza fase la diagnosi di tale limite motiva un redesign architetturale: **LineNet**, una rete che incorpora la geometria del gioco tramite uno scorer di linea condiviso ed equivarianza D_4 esatta nei pesi. LineNet con n -step= 3 raggiunge **61,29** (seme 42) e **63,19** (seme 7) su valutazione hold-out, entrambi confermati per mezzo dell'API ufficiale del gioco, superando la soglia di 60 con margine. Il documento riporta sia i risultati positivi sia quelli negativi, ciascuno motivato da dati numerici.

Contents

1	Introduzione	3
1.1	Il gioco	3
1.2	Obiettivo e criteri di valutazione	3
1.3	Ambiente di gioco e ambiente di addestramento	3
1.4	Struttura del codice	3
2	Stato iniziale e audit del codice	4
3	Metodologia	4
4	Le architetture di rete	4
4.1	MLP semplice	4
4.2	Rete dueling	5
4.3	LineNet	5
4.4	Rappresentazione dello stato	5
5	Studio di ablazione	5
5.1	Punto di partenza	5
5.2	Ritorni a n passi	5
5.3	Feature aggiuntive con i punteggi correnti di linea	6
5.4	Prioritized experience replay	6
5.5	Data augmentation per simmetria D_4	6
5.6	Decadimento del learning rate	7

5.7	Capacità della rete e robustezza al seme	7
5.8	Riepilogo	7
6	Redesign architetturale: LineNet	7
6.1	Diagnosi del limite	7
6.2	Architettura	8
6.3	Correttezza e integrazione	8
6.4	Risultati	8
7	Configurazione finale e verifica	9
7.1	Comportamento sotto modifica dei punteggi	9
8	Conclusioni	10

1 Introduzione

1.1 Il gioco

Knister si gioca su una griglia 5×5 inizialmente vuota. A ogni turno vengono lanciati due dadi a sei facce e se ne somma il risultato, ottenendo un valore compreso tra 2 e 12; tale valore deve essere collocato in una cella libera. La partita dura sempre esattamente 25 turni e termina quando la griglia è piena. Poiché le celle non possono essere sovrascritte, l'agente non sceglie quale valore ricevere ma solo dove collocarlo: si tratta quindi di un problema di controllo in condizioni di incertezza.

Il punteggio viene calcolato in modo indipendente su 12 linee (5 righe, 5 colonne e 2 diagonali), sommando i punti della migliore combinazione presente in ciascuna: cinquina 10, poker 6, full house 8, doppia coppia 3, tris 3, coppia 1, scala contenente il 7 pari a 8 e scala che non lo contiene pari a 12. Le due diagonali sono pesate con un moltiplicatore fisso pari a 2. La somma di due dadi non ha distribuzione uniforme: il 7 è il valore più probabile, il che spiega perché una scala che lo evita valga più di una che lo include.

1.2 Obiettivo e criteri di valutazione

L'obiettivo consiste nell'addestrare un agente basato su Deep Q-Network che massimizzi il punteggio medio. La griglia di valutazione prevede 2 punti per un punteggio medio maggiore o uguale a 60, 1,5 punti per 45, 1 punto per 30 e 0,5 punti per 20, con una tolleranza di 2 punti sulle soglie. In sede orale può essere richiesto di modificare i punteggi delle combinazioni, per verificare la capacità di generalizzazione della rete senza ripetere l'addestramento; il comportamento dell'agente in questa condizione è discusso nella Sezione 7.1.

1.3 Ambiente di gioco e ambiente di addestramento

Viene fornita una classe `KnisterGame` (modulo `api.py`) che implementa il ciclo di vita della partita. Il reward restituito a ogni mossa è il delta incrementale di punteggio prodotto dalla mossa stessa.

Data la scala richiesta dall'addestramento (milioni di episodi), è stato sviluppato un ambiente vettorizzato equivalente, `FastVectorKnister`, che esegue fino a 1024 partite in parallelo tramite operazioni NumPy anziché un ciclo Python per ogni mossa. La sua equivalenza all'implementazione ufficiale è verificata da `validate_fast_environment()`, eseguita all'avvio di ogni addestramento: la funzione istanzia altrettante partite di `KnisterGame`, applica le stesse azioni con gli stessi lanci e solleva un'eccezione alla prima discrepanza di reward, griglia, flag di terminazione o punteggio totale. L'addestramento e le valutazioni periodiche usano l'ambiente veloce; la verifica finale di un checkpoint avviene invece con l'API ufficiale (Sezione 7).

1.4 Struttura del codice

Il progetto è organizzato in file con responsabilità distinte:

- `api.py` — l'implementazione del gioco, non modificata.
- `train_knister_v2.py` — il motore: ambiente vettorizzato, replay buffer, encoder di stato, ciclo di addestramento Double DQN, procedure di valutazione, ritorni a n passi e tutte le varianti dello studio di ablazione, ciascuna dietro un flag disattivato di default.
- `linenet.py` — l'architettura LineNet (Sezione 6), il relativo encoder di stato e quattro validatori di correttezza eseguiti all'avvio.
- `linenet_train.py` — l'entrypoint che innesta LineNet nel ciclo di addestramento del motore senza modificarne il codice.

- `eval_tools.py` — strumenti di valutazione di un checkpoint: verifica sull'API ufficiale, ispezione, ricerca a un passo e test di generalizzazione.

La scelta di isolare LineNet in file separati preserva l'integrità del motore e consente di eseguire i confronti con l'architettura precedente sul codice originale, rendendo attribuibile alla sola architettura la differenza misurata.

2 Stato iniziale e audit del codice

Il punto di partenza è un agente Double DQN con testa dueling, encoder a 184 feature e $\gamma = 1$. Una revisione del codice ha confermato la correttezza dell'ambiente vettorizzato, del masking nel Double DQN e della scelta $\gamma = 1$, giustificata dal fatto che con reward incrementale e orizzonte fisso il ritorno non scontato coincide esattamente con il punteggio finale meno quello già accumulato, allineando la quantità stimata dalla rete alla metrica di valutazione. Sono però emersi due difetti.

Tabella dei punteggi duplicata. I valori delle combinazioni erano scritti come costanti in due punti indipendenti del codice, senza alcun legame con `api.py`. Una modifica dei punteggi avrebbe lasciato tali copie silenziosamente disallineate. La correzione consiste nel leggere la tabella una sola volta dalle costanti pubbliche di `KnisterGame` e nel passarla come parametro esplicito ovunque venga usata.

Aggregazione dueling non mascherata. La testa dueling calcola $Q = V + A - \bar{A}$, dove $\bar{A}$ è la media delle advantage. L'implementazione originale calcolava tale media su tutte e 25 le azioni, incluse le celle già occupate. Un test isolato ha confermato che la retropropagazione da una singola azione valida produceva comunque un contributo di gradiente non nullo su tutte le 25 uscite. La correzione limita la media alle sole azioni valide, tramite una maschera esplicita.

3 Metodologia

Tutti gli esperimenti condividono lo stesso protocollo: Double DQN, rete target aggiornata ogni 4000 aggiornamenti di gradiente, replay uniforme da 2 milioni di transizioni, batch da 1024, 2 aggiornamenti per round vettoriale di 1024 ambienti paralleli, $\gamma = 1$ ed ϵ decrescente linearmente da 1,0 a 0,03 sulla prima metà dell'addestramento.

La valutazione avviene su due livelli. Durante l'addestramento, ogni 100.000 episodi viene eseguita una valutazione greedy su 500 partite con seme fisso, e viene conservato come *best* il checkpoint con la media più alta. Al termine, tale checkpoint viene sottoposto a una valutazione hold-out finale su 5000 o 10.000 partite con un seme diverso, così da ottenere una stima meno soggetta al rumore e all'effetto di selezione. Tutte le valutazioni usano una politica greedy pura: l'esplorazione riguarda esclusivamente l'addestramento.

Ogni variante è stata testata isolatamente a scala ridotta (600.000 episodi) prima di un eventuale investimento a scala piena. Come mostrato nella Sezione 5.3, questo protocollo ha un limite: un vantaggio misurato a scala ridotta non garantisce lo stesso vantaggio a scala piena.

4 Le architetture di rete

Poiché il confronto tra architetture è il tema centrale del lavoro, questa sezione ne descrive la struttura interna.

4.1 MLP semplice

La variante più elementare: il vettore di stato attraversa due strati completamente connessi con attivazione ReLU e uno strato di uscita che produce 25 valori, uno per azione. Non esiste alcuna separazione tra il valore dello stato e la qualità relativa delle azioni.

4.2 Rete dueling

La variante usata in tutti gli esperimenti sull'architettura completamente connessa. Il vettore di stato attraversa un tronco condiviso di due strati con ReLU, dopo il quale il flusso si divide in due teste:

- la testa *valore* produce un singolo numero $V(s)$, che rappresenta la qualità complessiva dello stato indipendentemente dall'azione scelta;
- la testa *advantage* produce 25 numeri $A(s, a)$, uno per azione, che rappresentano quanto ciascuna azione sia migliore o peggiore della media.

Le due uscite vengono ricombinate come

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|\mathcal{A}_{\text{valide}}|} \sum_{a' \in \mathcal{A}_{\text{valide}}} A(s, a')$$

dove la media è calcolata sulle sole azioni valide, secondo la correzione descritta nella Sezione 2. La motivazione della separazione è che in molti stati la qualità complessiva domina la scelta specifica: la rete può apprendere V senza dover stimare con precisione ogni singola A .

4.3 LineNet

Descritta in dettaglio nella Sezione 6. La differenza sostanziale rispetto alle precedenti è che le uscite non sono prodotte da uno strato indipendente per azione, ma da moduli i cui pesi sono *condivisi* tra le 12 linee e tra le 25 celle.

4.4 Rappresentazione dello stato

Le prime due architetture ricevono il vettore **engineered184**, composto da: 25 valori normalizzati delle celle, 25 flag di validità, 25 reward immediati (uno per azione), 75 proiezioni del punteggio di riga, colonna e diagonale dopo ciascuna azione, 11 valori one-hot per il lancio corrente, 1 indicatore di progresso della partita e 22 valori di istogramma dei valori presenti e attesi. Si tratta di feature calcolate esternamente alla rete.

LineNet riceve invece solo 34 numeri grezzi (25 celle, 1 lancio, 8 valori della tabella dei punteggi): le proiezioni non servono in ingresso perché la rete le calcola internamente. Includere la tabella dei punteggi tra gli ingressi rende inoltre la rete condizionata sui punteggi anziché vincolata a valori fissi.

5 Studio di ablazione

5.1 Punto di partenza

Dopo le correzioni descritte nella Sezione 2, la configurazione di partenza (nessuna variante attiva, $n\text{-step}=1$) raggiunge $47,08 \pm 7,85$ su 10.000 partite hold-out a 3 milioni di episodi.

5.2 Ritorni a n passi

Con $\gamma = 1$ il ritorno a n passi è la somma semplice degli n reward reali successivi, troncata in prossimità della fine dell'episodio. L'ipotesi alla base è che l'apprendimento a singolo passo propaghi troppo lentamente l'informazione relativa a mosse che predispongono combinazioni completabili solo diversi turni dopo.

Il passaggio da $n = 1$ a $n = 3$ produce il guadagno maggiore osservato, mentre $n = 5$ aggiunge poco: incrementare ulteriormente la finestra comporta infatti l'inclusione nel target di un maggior numero di passi di comportamento esplorativo, che a inizio addestramento è quasi

Table 1: Ritorni a n passi, confronto isolato a 600.000 episodi.

Configurazione	Punteggio	Nota
$n = 1$	$\sim 43-44$	riferimento
$n = 3$	$45,68 \pm 9,30$	guadagno maggiore dello studio
$n = 5$	$46,17 \pm 10,13$	+0,5 su $n = 3$, rendimento decrescente

casuale. Esteso a 4 milioni di episodi con $n = 3$ e nessun'altra variante attiva, il risultato è $58,90 \pm 9,42$ su 10.000 partite hold-out: oltre 11 punti di miglioramento rispetto al punto di partenza, ottenuti da una singola modifica.

5.3 Feature aggiuntive con i punteggi correnti di linea

Questa variante aggiunge 75 feature che espongono il punteggio corrente di riga, colonna e diagonale per ciascuna azione. A 600.000 episodi, in combinazione con $n\text{-step} = 3$, mostra un vantaggio netto ($47,10 \pm 9,67$ contro $45,68 \pm 9,30$). Esteso a 3 milioni di episodi, però, il vantaggio si inverte.

Table 2: $n\text{-step} = 3$ isolato e con feature aggiuntive, stesso seme, 3 milioni di episodi.

Episodio	$n\text{-step}$ solo	Con feature	Differenza
400.000	41,65	44,54	+2,89
1.600.000	47,31	50,80	+3,49
2.400.000	55,25	56,19	+0,94
2.500.000	56,59	55,53	-1,06
Hold-out finale	57,49	56,42	-1,07

Per circa l'ottanta per cento dell'addestramento la combinazione risulta in vantaggio, ma il rapporto si inverte nell'ultimo quinto. L'interpretazione più plausibile è che feature pre-elaborate accelerino l'apprendimento iniziale ma vincolino la rappresentazione interna, limitando il perfezionamento nelle fasi avanzate. Il risultato ha inoltre rilevanza metodologica: un'ablazione condotta esclusivamente a scala ridotta avrebbe portato alla conclusione opposta.

5.4 Prioritized experience replay

Il campionamento prioritizzato estrae dal replay le transizioni con probabilità proporzionale al TD-error, con correzione di importance sampling nella loss. In combinazione con $n\text{-step} = 3$ a 600.000 episodi il risultato è lievemente negativo ($45,30 \pm 8,96$ contro $45,68 \pm 9,30$). La variante non è stata estesa a scala piena.

5.5 Data augmentation per simmetria D_4

Il punteggio dipende unicamente dal multi-insieme di valori di ciascuna linea e non dalla posizione fisica occupata: le otto trasformazioni del gruppo diedrale del quadrato producono quindi transizioni ugualmente valide, come verificato empiricamente. Applicando una trasformazione casuale per ciascun campione del minibatch, in combinazione con $n\text{-step} = 3$, il risultato a 600.000 episodi peggiora ($44,70 \pm 8,83$). Una spiegazione plausibile è che l'augmentation costringa la rete ad apprendere una funzione genuinamente invariante anziché sfruttare regolarità legate all'orientamento, rallentando la convergenza iniziale. Questa osservazione, insieme al risultato della Sezione 5.7, motiva il redesign descritto nella Sezione 6.

5.6 Decadimento del learning rate

Un primo tentativo, con decadimento attivo fin dal primo aggiornamento, ha prodotto un netto peggioramento ($54,47 \pm 9,61$ contro $58,90$): il confronto tra le due traiettorie ha mostrato che il learning rate era già sceso sotto il 40% per cento del valore iniziale proprio nella finestra tra 1,6 e 2,4 milioni di episodi, in cui la configurazione senza decadimento otteneva il guadagno maggiore. Introducendo un ritardo prima dell’inizio del decadimento (learning rate pieno fino al 75% per cento dell’addestramento) il risultato diventa statisticamente indistinguibile dal riferimento ($58,90 \pm 9,46$). Se ne conclude che il rallentamento osservato nella fase finale non dipende dall’ampiezza del passo di ottimizzazione.

5.7 Capacità della rete e robustezza al seme

Aumentando la larghezza degli strati nascosti da 384 a 512 unità si osserva un vantaggio a 600.000 episodi ($47,11 \pm 10,18$), ma a scala piena il quadro è meno netto: lungo la traiettoria la rete più ampia risulta quasi sempre pari o in vantaggio, mentre sulla valutazione hold-out finale è leggermente inferiore ($58,41 \pm 9,20$ contro $58,90$). Una ripetizione con seme 7 dà $58,15 \pm 9,19$. I tre risultati a scala piena — $58,90$, $58,41$ e $58,15$ — rientrano in un intervallo di 0.75 punti: il valore prossimo a 58 – 59 costituisce quindi un limite stabile e riproducibile per questa famiglia di architetture, non l’esito di un’inizializzazione favorevole.

5.8 Riepilogo

Table 3: Riepilogo delle varianti testate (600.000 episodi salvo indicazione).

Variante	Punteggio	Esito
Riferimento (n -step= 1)	~ 43 – 44	punto di partenza
n -step= 3	$45,68 \pm 9,30$	positivo, netto
n -step= 5	$46,17 \pm 10,13$	positivo, marginale
Feature per linea	$47,10 \pm 9,67$	si inverte a scala piena
Prioritized replay	$45,30 \pm 8,96$	lievemente negativo
Simmetrie D_4	$44,70 \pm 8,83$	negativo
Learning rate decrescente (4M)	$58,90 \pm 9,46$	neutro
Rete a 512 unità (4M)	$58,41$ / $58,15$	neutro

Nessuna delle cinque varianti diverse dai ritorni a n passi ha prodotto un miglioramento robusto. La configurazione finale dell’architettura completamente connessa resta quindi la più semplice, e il suo risultato ($58,90 \pm 9,42$) è stato verificato per mezzo dell’API ufficiale, ottenendo $58,86 \pm 9,52$ su 10.000 partite: una differenza di 0,04 punti rispetto all’ambiente vettorizzato.

6 Redesign architetturale: LineNet

6.1 Diagnosi del limite

Tre osservazioni dello studio di ablazione convergono su un’unica spiegazione: l’aumento di capacità risulta neutro (Sezione 5.7), la data augmentation D_4 risulta dannosa (Sezione 5.5) e il punteggio si assesta attorno a 58 – 59 indipendentemente dal seme. Il fattore comune è la *forma* della rete: un percettrone multistrato applicato a un vettore piatto non possiede alcuna struttura che rispecchi la geometria del gioco, ossia 12 linee alle quali si applica la medesima logica di combinazione. La rete deve quindi riapprendere tale regolarità dai dati — il che rende la data augmentation un ostacolo anziché un aiuto — e la capacità aggiuntiva non serve, poiché il problema non risiede nel numero di parametri ma nell’assenza di condivisione tra di essi.

6.2 Architettura

LineNet incorpora questa struttura direttamente nei pesi. Il punteggio si fattorizza come

$$\text{Punteggio totale} = \sum_{\ell=1}^{12} w_{\ell} \cdot f(\text{valori della linea } \ell)$$

con f identica per tutte le linee e w_{ℓ} pari a 2 per le diagonali e 1 altrimenti. La rete rispecchia tale fattorizzazione tramite uno *scorer di linea condiviso*: una rete di due strati applicata con gli stessi pesi a ciascuna delle 12 linee, così che quanto appreso valutando una riga si trasferisca automaticamente a ogni colonna e diagonale.

Il flusso in avanti si articola in cinque passaggi. Le 12 linee vengono estratte dalla griglia; ciascuna viene valutata con la routine di scoring del motore, la stessa usata dall’ambiente. Per ognuna delle 60 incidenze cella-linea (ogni cella appartiene a 2, 3 o 4 linee, per un totale di 60 coppie) viene calcolato il punteggio ipotetico risultante dall’inserimento del valore corrente, ottenendo per differenza il guadagno esatto della mossa. I contributi relativi a celle già occupate vengono azzerati e quelli rimanenti vengono aggregati per azione. Un vettore di contesto globale, ottenuto da media e massimo degli embedding di linea insieme ad alcune statistiche di partita, alimenta la testa valore, mentre la testa advantage è condivisa sulle 25 celle. La ricombinazione dueling usa una media mascherata sulle sole azioni valide, strutturalmente e non per configurazione.

Ne consegue che la rete non deve *apprendere* il reward immediato: lo *calcola* esattamente. Ciò che apprende è il valore strategico a lungo termine, ovvero quali configurazioni predispongono combinazioni future.

Due proprietà rendono l’architettura corretta per costruzione:

- **Equivarianza D_4 esatta.** Le feature non contengono alcuna identità di riga, colonna o posizione, ma soltanto il flag di diagonale e il peso. Ne segue che per ogni griglia g e ogni simmetria T vale $Q(T(g)) = P_T(Q(g))$, dove P_T è la permutazione delle azioni indotta da T . A differenza della data augmentation, che tenta di *insegnare* l’invarianza dai dati, qui essa è garantita dalla struttura dei pesi. La verifica numerica su tutte e otto le simmetrie dà un errore inferiore a 2×10^{-4} , riconducibile alla sola precisione in virgola mobile.
- **Coerenza di scoring.** Il calcolo dei punteggi riusa direttamente la routine dell’ambiente, senza reimplementazioni, garantendo che il valore attribuito a una mossa coincida con quello assegnato dal gioco. La verifica su 4800 mosse dà un errore inferiore a 10^{-3} .

Con le dimensioni predefinite la rete conta 110.786 parametri, meno della variante completamente connessa a 512 unità: il guadagno deriva quindi dalla struttura e non dalla dimensione.

6.3 Correttezza e integrazione

L’architettura è validata da quattro test automatici eseguiti a ogni avvio: correttezza dell’encoding, decomposizione del reward, equivarianza D_4 e finitezza dei gradienti. L’integrazione riusa il ciclo di addestramento del motore senza modificarlo, garantendo che protocollo di acting, replay, ritorni a n passi e selezione del checkpoint migliore siano identici a quelli degli esperimenti precedenti.

6.4 Risultati

Due esperimenti separano il contributo dell’architettura da quello dei ritorni a n passi.

Con $n\text{-step}=1$ e 3 milioni di episodi LineNet raggiunge $58,74 \pm 9,70$: la sola architettura eguaglia dunque il miglior risultato dell’architettura precedente, che richiedeva $n\text{-step}=3$.

Con $n\text{-step}=3$ a 600.000 episodi raggiunge $53,30 \pm 10,53$, contro $45,68 \pm 9,30$ dell'architettura precedente nelle stesse condizioni: un vantaggio di 7,6 punti attribuibile alla sola architettura. I due contributi si sommano senza interferenza.

Table 4: Configurazione finale: LineNet con $n\text{-step}=3$ e con 3 milioni di episodi.

Seme	Hold-out	Verifica con API ufficiale
42	$61,29 \pm 10,30$	61,48
7	63,19	63,13 / 63,15 / 63,27

Entrambi i semi superano la soglia di 60 con margine e la verifica diretta conferma i valori. La coerenza tra due addestramenti indipendenti indica che il superamento della soglia è una proprietà stabile dell'architettura.

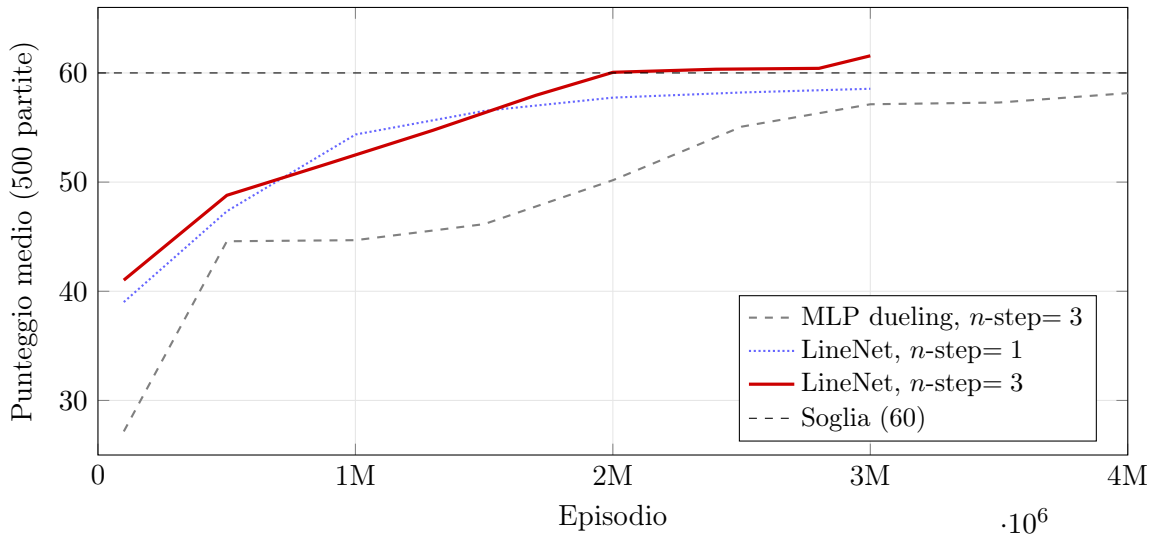


Figure 1: Confronto tra le tre configurazioni principali. L'architettura e i ritorni a n passi contribuiscono in modo indipendente: LineNet con $n\text{-step}=1$ raggiunge il livello dell'architettura precedente con $n\text{-step}=3$, e la combinazione delle due supera la soglia.

7 Configurazione finale e verifica

La configurazione definitiva è costituita dall'architettura LineNet con ritorni a 3 passi, addestrata per 3 milioni di episodi. Il risultato migliore si ottiene con il seme 7: 63,19 su valutazione hold-out, confermato per mezzo dell'API ufficiale su tre semi di valutazione distinti (63,13, 63,15 e 63,27). Il checkpoint ottenuto con il seme 42 (61,29) viene conservato come conferma di riproducibilità.

La verifica con l'API ufficiale carica il checkpoint, ricostruisce la rete e istanzia direttamente le partite di **KnisterGame**: la rete si limita a scegliere l'azione, mentre generazione dei lanci, aggiornamento della griglia e calcolo del punteggio sono interamente gestiti dall'implementazione fornita.

7.1 Comportamento sotto modifica dei punteggi

Tenendo conto che i punteggi delle combinazioni possono essere modificati, il comportamento dell'agente è stato verificato anche in questa condizione. LineNet riceve la tabella dei punteggi tra i propri ingressi, ed è quindi in grado di adattarsi senza riaddestramento.

Table 5: Effetto di una modifica della tabella (tris da 3 a 5 punti), 2000 partite, senza riaddestramento. Il valore sulla tabella originale (62,66) differisce da quello riportato nella Tabella 4 perché ottenuto su un campione diverso (2000 partite con seme dedicato, anziché 10.000): la differenza rientra nella variabilità campionaria.

Condizione	Punteggio
Tabella originale	$62,66 \pm 10,21$
Tabella modificata, rete informata della modifica	$65,90 \pm 9,72$
Tabella modificata, rete non informata	$65,54 \pm 10,04$

Il punteggio non degrada: la strategia appresa — costruire linee con combinazioni di valore — resta valida quando i valori cambiano, e l’agente continua a operare correttamente anche senza essere informato della modifica. Va osservato che la rete non è stata addestrata con tabelle variabili, pur essendo tale modalità implementata nel motore; la robustezza osservata deriva dunque dalla natura della strategia appresa e non da un addestramento specifico.

8 Conclusioni

Il lavoro individua due leve indipendenti e cumulabili. La prima, algoritmica, sono i ritorni a n passi con $n = 3$, unica variante dello studio di ablazione a produrre un miglioramento robusto, che porta l’architettura completamente connessa da 47,08 a 58,90. La seconda, architetturale, è LineNet, motivata dalla diagnosi del limite raggiunto dalla precedente architettura e basata sull’incorporazione della geometria del gioco tramite scorer di linea condiviso ed equivarianza D_4 esatta. La combinazione delle due raggiunge 63,19 e 61,29 su due semi indipendenti, entrambi verificati con l’API ufficiale.

Il percorso segue due principi applicati in modo coerente: ogni scelta è motivata da un confronto numerico e non da congettura, e i risultati negativi sono riportati con lo stesso dettaglio di quelli positivi. Tra questi ultimi merita particolare attenzione il caso delle feature aggiuntive, la cui conclusione a scala ridotta si è rivelata opposta a quella valida alla scala di impiego: un risultato che ha condizionato il metodo adottato nel resto del lavoro.